

Network Telemetry & Real-Time Network Performance Monitoring System

with Real-Device Validation using GNS3 / VyOS

Vadithya Harsha Vardhan Nayak

Roll No: 23110349

IIT Gandhinagar

23110349@iitgn.ac.in

Sailaja Mudu

Roll No: 23110288

IIT Gandhinagar

23110288@iitgn.ac.in

Banoth Arvind

Roll No: 23110058

IIT Gandhinagar

23110058@iitgn.ac.in

Abstract—This project implements a modular Network Telemetry and Real-Time Network Performance Monitoring System, conceptually aligned with RFC 9232, that collects bandwidth, utilization, latency, packet-loss, and device-availability metrics and presents them through live dashboards with threshold-based alerting. The system supports two interchangeable telemetry sources behind a single common interface: a deterministic custom simulator, and a real SNMPv2c collector for physical or virtual devices. Telemetry is transported using gRPC and Protocol Buffers, persisted in PostgreSQL, and visualized in Grafana. A threshold-based alert engine with deduplication and automatic recovery detection raises alerts for high bandwidth utilization, high latency, packet loss, and device unavailability. The SNMP collection pipeline was additionally validated against a real (non-simulated) virtual router built with GNS3 and VyOS inside an isolated VirtualBox host-only network, and the complete system was verified with an automated suite of 112 test cases (0 failed, 0 skipped).

Index Terms—Network Telemetry, RFC 9232, SNMP, gRPC, Protocol Buffers, PostgreSQL, Grafana, GNS3, VyOS, Threshold Alerting, Real-Time Monitoring.

I. INTRODUCTION

Network administrators need continuous visibility into bandwidth utilization, latency, packet loss, and device availability to detect and respond to performance degradation before it impacts users. Manually polling devices or reading raw counters is impractical at scale, and off-the-shelf commercial monitoring suites are often too heavyweight or closed-source for an academic setting where the goal is to understand and explain every layer of the pipeline. This project builds a complete, end-to-end telemetry pipeline from first principles: raw SNMP counters in, calculated performance metrics and threshold-based alerts out, visualized in near real time.

A. Objectives

- Collect standardized telemetry (bandwidth, utilization, latency, packet loss, device availability) from network devices.
- Support a deterministic custom simulator and real SNMPv2c polling of physical/virtual devices behind one common interface.
- Transport telemetry efficiently using gRPC and Protocol Buffers.
- Persist historical telemetry in PostgreSQL with an indexed, query-efficient schema.
- Visualize live and historical telemetry in Grafana with device/interface filtering.
- Implement threshold-based alerting with deduplication and automatic recovery detection.
- Validate the SNMP collector against a real, non-simulated device using GNS3/VyOS.
- Verify correctness with an automated pytest suite covering unit, component, and integration levels.

B. Scope

The system is an academic demonstration of network telemetry concepts aligned with RFC 9232, not a production-grade commercial product. SNMPv3 support, a durable message queue, and multi-tenant authentication are explicitly out of scope and documented as future enhancements.

II. TOOLS & TECHNOLOGIES

Table 1: Core technology stack

Category	Technology
Language	Python 3.12
Metric Collection	SNMPv2c (pysnmp)
Latency Measurement	ICMP (OS ping via subprocess)
Secondary Source	Custom Python simulator
Transport	gRPC + Protocol Buffers
Backend Framework	FastAPI (async, OpenAPI docs)
Database	PostgreSQL (SQLAlchemy ORM)
Visualization	Grafana
Containerization	Docker / Docker Compose
Testing	pytest
Virtual Network Lab	GNS3 + VyOS / Linux snmpd

III. SYSTEM ARCHITECTURE

Two interchangeable telemetry sources feed the same collector pipeline: a custom Python simulator (for demos) and a real SNMPv2c source (for physical or virtual devices). Both implement the same `MetricSource` interface and produce the same sample data shape, so nothing downstream (metric calculation, transport, storage, alerting, visualization) can distinguish which one produced a given sample.

A. Data Flow

Network Device / Simulator → raw counters collected once per poll interval → *Python Collector* converts raw counters into bandwidth/utilization by differencing two consecutive polls using actual measured elapsed time → *gRPC + Protobuf* batches and transports records → *FastAPI Backend* validates, ingests, and triggers alert evaluation → *PostgreSQL* persists devices, metrics and alerts → *Grafana* queries PostgreSQL directly for near-real-time dashboards.

B. RFC 9232 Alignment

Table 2: Conceptual alignment with RFC 9232

RFC 9232 Concept	Project Component
Telemetry Source	Router/switch (SNMP) or simulator
Telemetry Collector	collector/ — polls, standardizes, forwards
Transport / Encoding Consumer / Analytics	gRPC + Protocol Buffers backend/ — FastAPI + PostgreSQL
Historical Storage	PostgreSQL
Visualization	Grafana

IV. IMPLEMENTATION

A. Simulator & SNMP Collector

The simulator generates realistic cumulative interface counters (not precomputed bandwidth) across six deterministic scenarios — `NORMAL`, `HIGH_BANDWIDTH`, `HIGH_LATENCY`, `PACKET_LOSS`, `DEVICE_DOWN`, `RECOVERY` — enabling on-demand demonstration of alert conditions. The SNMP source uses SNMPv2c via `pysnmp`, polling MIB-2/IF-MIB OIDs (`sysUpTime`, `ifName`, `ifHCInOctets/ifHCOutOctets`, `ifHighSpeed`, `ifOperStatus`, packet and error counters). Community strings are read exclusively from environment variables, and a device must be explicitly marked `authorized: true` before it is ever polled.

B. Bandwidth & Utilization Engine

Implemented as pure, unit-testable functions:

$$\text{bandwidth (bps)} = \frac{\Delta \text{bytes} \times 8}{\text{elapsed seconds}}, \quad \text{util. \%} = \frac{\text{bandwidth}}{\text{iface speed}} \times 100$$

Counter resets (reboot) are distinguished from counter wraparound using a plausibility check on the implied bandwidth of the wraparound-corrected delta.

C. Transport, Backend & Storage

A single `TelemetryBatch` protobuf message carries all metrics for one device per poll. The gRPC client retries with exponential backoff and never crashes the collector when the backend is unavailable. The FastAPI backend is layered as API router → service → repository, exposing

`/health`, `/devices`, `/metrics`, `/metrics/export` (CSV), and `/alerts` endpoints. PostgreSQL stores `devices`, `telemetry_metrics`, and `alerts`, with composite indexes on `(device_id, timestamp)` and `(device_id, metric_type, timestamp)`.

D. Alert Engine & Dashboards

Threshold-based alerting is keyed on `(device_id, interface, alert_type, metric_type)` — a metric breaching threshold for many consecutive polls produces exactly one `ACTIVE` row, auto-transitioned to `RESOLVED` on recovery. Four Grafana dashboards (Network Overview, Bandwidth, Latency, Alerts) read PostgreSQL directly with a 5–10s refresh.

V. GNS3 / VYOS REAL-DEVICE INTEGRATION

The custom simulator proves the pipeline’s logic but not that the SNMP collector works against a genuinely independent, real network device. A virtual router environment was built using GNS3 with a VyOS/Linux SNMP agent, proving the same, already-unit-tested `SNMPSource` implementation against real SNMP traffic on the wire, with zero code changes.

A. Setup Summary

- Dedicated host-only NIC on the GNS3 VM, bridged into the topology via a Cloud node onto the `192.168.56.0/24` lab subnet.
- `snmpd` enabled on the target device with a read-only community string restricted to the lab subnet, verified externally with `snmpwalk` before wiring into the project.
- Device added to a separate `devices-gns3.yaml` with `authorized: true`; the original simulator’s `devices.yaml` left untouched.
- A second, independent collector process (same unmodified `collector/main.py`) run in `SNMP` mode alongside the `simulator-mode` collector, both feeding the same backend.

Real interface counters collected over SNMPv2c from the virtual device were successfully ingested through the exact same pipeline used for the simulator, and displayed side-by-side with simulated telemetry in Grafana.

VI. TESTING

A. Test Strategy

Tests are organized by layer: pure unit tests for calculation logic (no I/O), mocked component tests for SNMP/gRPC/-database (SQLite in-memory stands in for PostgreSQL), and true integration tests that bind a real gRPC socket end-to-end.

B. Automated Test Suite Summary

Total result: 112 passed, 0 failed, 0 skipped.

Table 3: Automated test suite coverage by area

Area	What It Validates
Core Contracts	MetricSource/LatencySource interfaces well-formed
SNMP Collector	OID construction, credential loading, discovery/polling, authorization gate
Bandwidth Engine	Normal/zero/high traffic, poll intervals, reset vs. wraparound
ICMP Latency	Win/Linux ping parsing, timeout, partial loss
Simulator	All 6 scenarios, monotonic counters, overrides
gRPC Transport	Serialization, retry/backoff, backend-unavailable handling
DB & Ingestion	Device/interface upsert, telemetry storage, alert invocation
Alert Engine	Threshold breach, deduplication, recovery, independent tracking
REST API	All endpoints, filtering, 404s, CSV export
Integration	Full pipeline over a real gRPC TCP socket

C. Representative Unit/Component Test Cases

D. Live/Manual Demonstration Test Cases

Beyond the automated suite, the alert engine’s behaviour was also demonstrated live against the running system using the `demo_scenario.py` script. Each case below lists the scenario name, the exact trigger command, the expected result, and the corresponding Grafana/terminal evidence.

TC-02 — High Bandwidth Threshold Breach. Scenario `HIGH_BANDWIDTH` set for device `router-01`. **Trigger:**

```
python scripts\demo_scenario.py -device router-01
-scanario HIGH_BANDWIDTH -rounds 3 -interval 3
```

Expected Result: RX/TX utilization exceeds the 80% threshold on both interfaces, raising a `HIGH_BANDWIDTH WARNING` alert.

Status: **PASS**

Time	Device ID	Interface	Alert Type	Severity	Current Value	Threshold	Status
2025-05-10 16:22:47	router-01	eth0	HIGH_BANDWIDTH	WARNING	90	80	ACTIVE
2025-05-10 16:22:48	router-01	eth1	HIGH_BANDWIDTH	WARNING	90	80	ACTIVE

Figure 1: TC-02 — Active `HIGH_BANDWIDTH` alerts for `router-01` (`eth0`, `eth1`).

Table 4: Representative automated test cases

#	Test Case	Input	Expected Result	St.
1	Normal counter increase	prev=1000B, curr=2000B, 10s	800 bps, valid	P
2	Counter wraparound	near $2^{32}-1 \rightarrow$ small, plausible	Detected, valid bw returned	P
3	Counter reset (re-boot)	Large decrease, implausible	Rejected as reset	P
4	Zero interface speed	speed=0	Utilization invalid	P
5	Unauthorized SNMP poll	authorized: false	PermissionError before send	P
6	SNMP timeout	Mocked no response	reachable=false, others unaffected	P
7	Alert deduplication	20 consecutive breaches	Exactly 1 ACTIVE row	P
8	Alert recovery	Metric returns below threshold	ACTIVE→RESOLVED, resolved_at set	P
9	Independent in/out alerts	util_in breaches, util_out ok	Tracked independently	P
10	Device down→recovery	DEVICE_DOWN then RECOVERY	Alert created, then resolved	P
11	gRPC backend unavailable	No backend listening	Backoff retry, drop & log, no crash	P
12	CSV export filtering	device/metric filters	Correct rows, headers	P
13	Unknown device lookup	GET /devices/does-not-exist	HTTP 404	P
14	E2E real-socket pipeline	HIGH_BANDWIDTH over live gRPC	Stored, alert created & resolves	P



Figure 2: TC-02 — Bandwidth and utilization spikes on the Grafana Bandwidth dashboard.

```
python scripts\demo_scenario.py -device router-01
-scanario NORMAL -rounds 2 -interval 3
```

Expected Result: The TC-02 alert flips `ACTIVE` → `RESOLVED`, with `resolved_at` populated.

Status: **PASS**

Started At	Resolved At	Device ID	Interface	Alert Type	Severity	Current Value	Threshold	Status
2025-05-10 16:22:47	2025-05-10 16:22:56	router-01	eth0	HIGH_BANDWIDTH	WARNING	90	80	RESOLVED

Figure 3: TC-03 — Alert history showing both `router-01` alerts `RESOLVED`.

TC-04 — High Latency Threshold Breach. Trigger:

```
python scripts\demo_scenario.py -device switch-01
-scanario HIGH_LATENCY -rounds 3 -interval 3
```

Expected Result: `latency_avg_ms` ≈ 150 (> 100 ms threshold) → `HIGH_LATENCY` alert, `WARNING`.

Status: **PASS**

TC-03 — Alert Recovery (Bandwidth). Trigger:

TC-05 — Packet Loss Threshold Breach. Trigger:

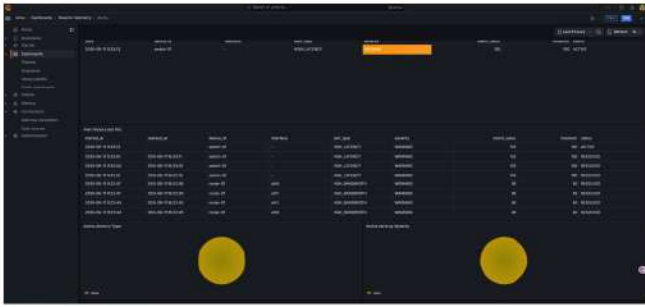


Figure 4: TC-04 — HIGH_LATENCY alert and alert history for switch-01.



Figure 5: TC-04 — Latency-over-time dashboard showing the breach and recovery spikes.

```
python scripts\demo_scenario.py -device switch-01
-scenario PACKET_LOSS -rounds 3 -interval 3
```

Expected Result: packet_loss_percent ≈ 15 ($> 5\%$ threshold) $\rightarrow$ PACKET_LOSS alert, WARNING.

Status: PASS

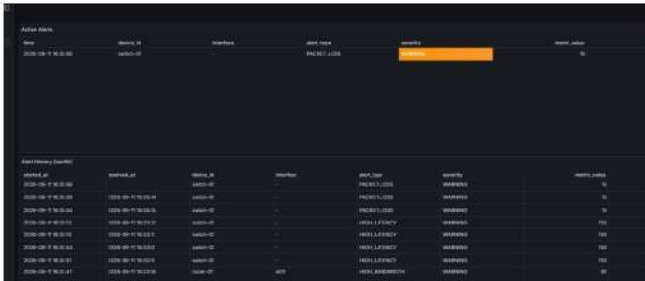


Figure 6: TC-05 — Active PACKET_LOSS alert and history for switch-01.

TC-06 — Device Down (Component Failure). Trigger:

```
python scripts\demo_scenario.py -device switch-02
-scenario DEVICE_DOWN -rounds 1
```

Expected Result: device_reachable = 0.0 $\rightarrow$ DEVICE_DOWN alert, severity = CRITICAL (the only CRITICAL case in the system).

Status: PASS

TC-07 — Device Recovery. Trigger:

```
python scripts\demo_scenario.py -device switch-02
-scenario RECOVERY -rounds 1
```

Expected Result: reachable flips back to true; the DEVICE_DOWN alert transitions to RESOLVED.



Figure 7: TC-05 — Packet-loss (%) spike on the Latency dashboard.

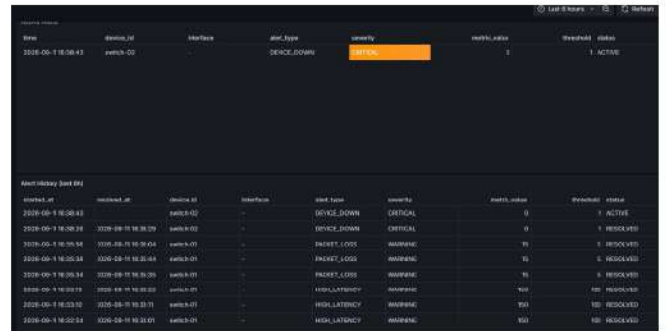


Figure 8: TC-06 — CRITICAL DEVICE_DOWN alert for switch-02.

Status: PASS



Figure 9: TC-07 — DEVICE_DOWN alert history showing RESOLVED status for switch-02.

TC-08 — Alert Deduplication Under Sustained Breach.

Trigger:

```
python scripts\demo_scenario.py -device router-02
-scenario HIGH_BANDWIDTH -rounds 8 -interval 2
```

Expected Result: 8 consecutive breaching polls produce exactly 1 ACTIVE alert row per interface, not 8.

Status: PASS

TC-09 — Backend/Component Unavailability (Resilience).

Trigger:

```
docker compose stop backend
```

Expected Result: Collector logs a gRPC send attempt ...UNAVAILABLE, retries with exponential backoff, then logs a dropped-batch error — the collector does not crash.

Status: PASS

VII. RESULTS

The completed system demonstrates real-time collection, storage, and visualization of network telemetry from two independent sources — the custom simulator and a real SNMP-pollled GNS3/VyOS virtual device — through one unmodified pipeline. All 112 automated tests pass, and all 8 manual demonstration scenarios (TC-02 – TC-09) produced the expected alert lifecycle (raise, deduplicate, resolve) exactly as de-

Interface	Severity	Alert Details
eth0	CRITICAL	High bandwidth usage detected on eth0
eth1	CRITICAL	High bandwidth usage detected on eth1

Figure 10: TC-08 — Deduplicated HIGH_BANDWIDTH alerts for router-02 (one row per interface).

```
PS C:\Users\saila\Downloads\network-telemetry-final (3)\network-
telemetry> docker compose stop backend
[+] stop 1/1
✓Container network-telemetry-backend-1 Stopped 0.5s
PS C:\Users\saila\Downloads\network-telemetry-final (3)\network-
telemetry> docker compose start backend
[+] start 2/2
✓Container network-telemetry-postgres-1 Healthy 0.5s
✓Container network-telemetry-backend-1 Started 0.3s
```

Figure 11: TC-09 — Backend stopped and restarted via `docker compose`; collector remains resilient.

signed, including the single CRITICAL-severity case (device unreachability).

VIII. CHALLENGES FACED & SOLUTIONS

- **Isolated virtual networks:** the GNS3 VM's management network and the in-project router topology were unconnected despite sharing a subnet — resolved by bridging a dedicated third NIC into the topology via a Cloud node.
- **Cloned VM interface renaming:** cloned VyOS images did not retain a fixed active interface — resolved by checking interface state via the router CLI before reapplying IP configuration.
- **snmpd configuration corruption:** repeated `sed`/heredoc edits corrupted `snmpd.conf` — resolved by retyping a minimal, verified-clean configuration directly.
- **PostgreSQL password mismatch:** the DB password is set only at first container startup — resolved by clearing the data volume and reinitializing.
- **Interface speed on virtual devices:** minimal SNMP agents may not populate `ifHighSpeed` — verified in advance with `snmpwalk`.
- **Python module resolution:** running helper scripts directly failed with `ModuleNotFoundError` — resolved by setting `PYTHONPATH` to the project root.

IX. SECURITY & SAFETY CONSIDERATIONS

- SNMP community strings are read exclusively from environment variables, never committed to YAML or source code.
- Each device requires an explicit `authorized: true` flag, enforced in code as a hard `PermissionError`, independent of the community-string requirement.

- The GNS3/VyOS lab network is isolated to a VirtualBox host-only adapter with no bridge to a physical/shared LAN.
- No physical departmental or third-party device was polled; all real-SNMP validation used a self-hosted virtual device under the author's own control.

X. CONCLUSION & FUTURE ENHANCEMENTS

This project delivers a complete, working network telemetry pipeline — SNMP/ICMP collection through gRPC transport, PostgreSQL storage, and Grafana visualization with threshold-based alerting — validated by 112 automated tests and by real SNMP data from an independent virtual device built in GNS3. The same, unmodified collector code correctly handled both the deterministic simulator and a real SNMP agent, demonstrating that the core design goal of a pluggable, interchangeable telemetry source was achieved in practice.

Planned enhancements include SNMPv3 support, a multi-hop GNS3 lab with multiple routed VyOS devices, a durable queue between collector and backend, webhook/email alert notification hooks, and a live scenario-switching endpoint for the simulator.

REFERENCES

- [1] RFC 9232 — Network Telemetry Framework, IETF.
- [2] RFC 2863 — The Interfaces Group MIB (IF-MIB).
- [3] RFC 1213 — Management Information Base for Network Management of TCP/IP-based internets (MIB-II).
- [4] pysnmp documentation.
- [5] gRPC and Protocol Buffers documentation, Google/CNCF.
- [6] FastAPI documentation.
- [7] Grafana documentation.
- [8] GNS3 documentation.
- [9] VyOS documentation.